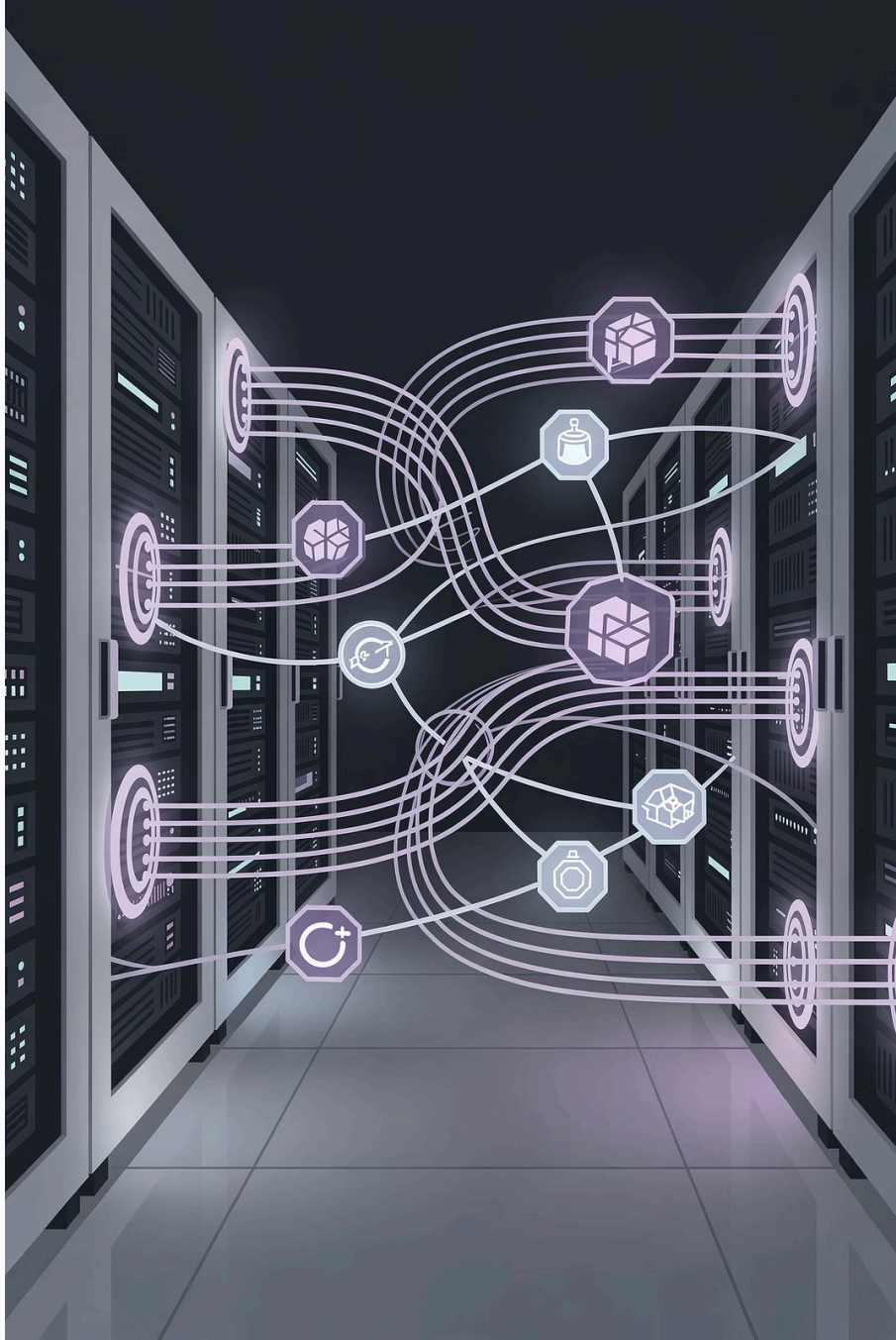
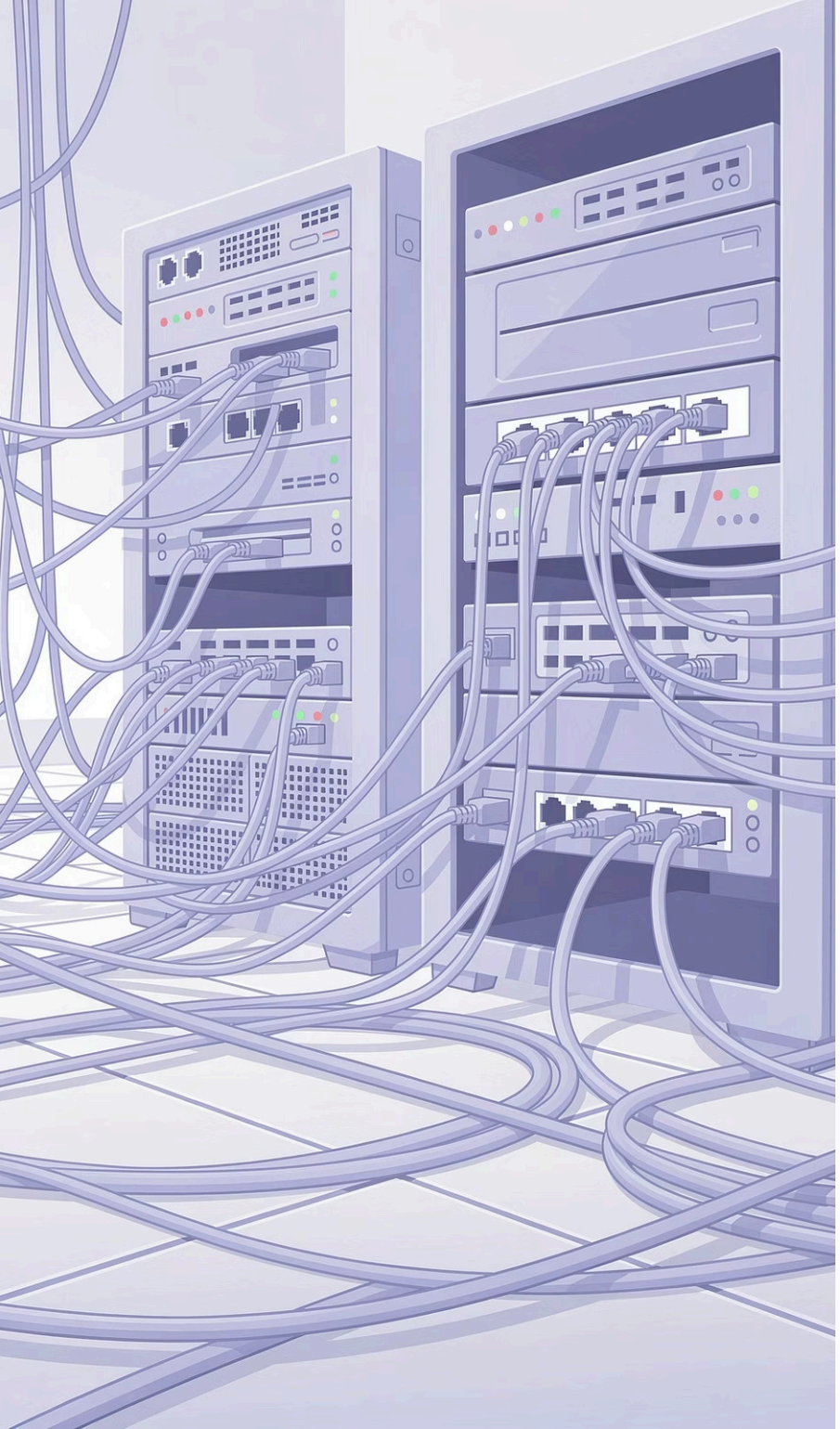
ML + Data Engineering Integration

Building production-grade systems where data pipelines and machine learning workflows operate as a single, cohesive platform — enabling faster iteration, better governance, and more reliable models.

ENGINEERING DEEP DIVE

PRODUCTION SYSTEMS





The Problem: Fragmented by Design

Most organisations build data pipelines and ML systems in isolation — different teams, different tools, different deployment cycles. This creates compounding pain points that slow down every model release.

Siloed Pipelines

Data engineering and ML teams operate in separate toolchains. Handoffs between them are manual, error-prone, and poorly documented — making reproducibility nearly impossible.

Maintenance Nightmare

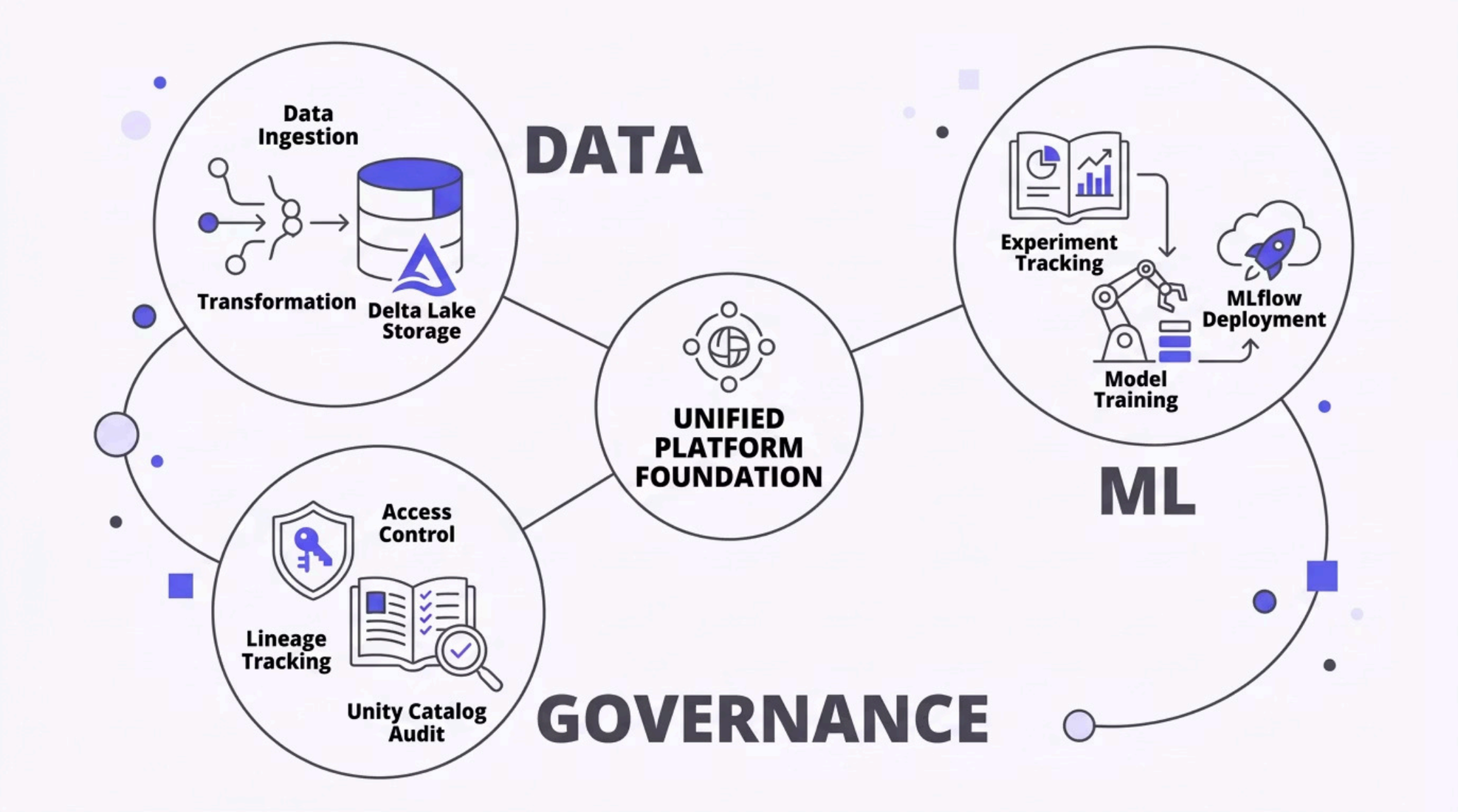
When pipelines and models are decoupled, a schema change upstream can silently break a model in production. There's no single source of truth, and debugging requires context spread across multiple systems.

Slow Iteration Cycles

Re-engineering data transforms for every new model experiment wastes weeks. Teams end up duplicating logic rather than sharing reusable, versioned assets.

The Unified Platform Vision

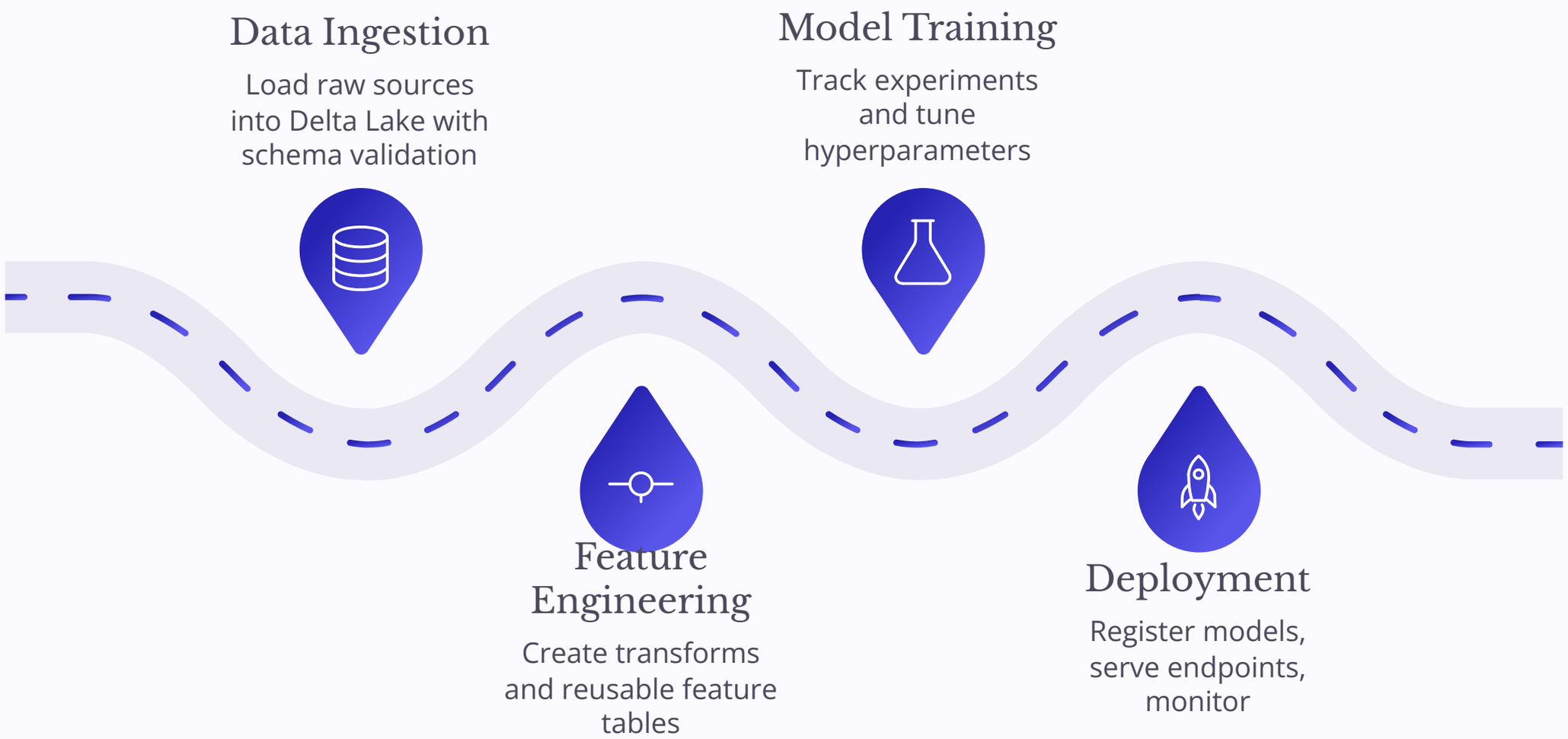
The answer is a single, integrated platform where data, ML, and governance are first-class citizens — sharing storage, compute, and metadata to eliminate silos entirely.



When data contracts, feature definitions, and model registries live in the same ecosystem, every team — data engineers, ML engineers, and analysts — can collaborate without costly context-switching.

The End-to-End ML Workflow

A production ML system follows a repeatable four-stage lifecycle. Each stage has clear ownership, tooling, and quality gates — and each feeds directly into the next.



Understanding this lifecycle end-to-end is critical for both data engineers and ML engineers — the boundaries between roles blur deliberately at each stage handoff.

The Role of Data Engineering

Data engineers are the unsung architects of every successful ML system. Their work determines whether a model trains on reliable, consistent data — or on noise.

What Data Engineers Own

Data engineers are responsible for the infrastructure that makes ML possible: clean, validated, versioned data arriving on schedule and at the right granularity.

Core Responsibilities

- **Clean Data** — Schema enforcement, null handling, deduplication, and anomaly detection before data reaches feature pipelines
- **Feature Pipelines** — Building reusable, tested transformations that multiple models can consume without re-implementation
- **Data Quality** — Automated expectations (e.g. Great Expectations, Delta constraints) that catch drift and failures early
- **Lineage** — Tracking data provenance so model behaviour can be traced back to source records

Introduction to MLflow

MLflow is the open-source backbone of ML lifecycle management. It provides the primitives every production ML system needs — without locking you into a single cloud or framework.

Experiment Tracking

Log parameters, metrics, and artefacts for every training run. Compare experiments visually to understand what drives model performance.

Model Versioning

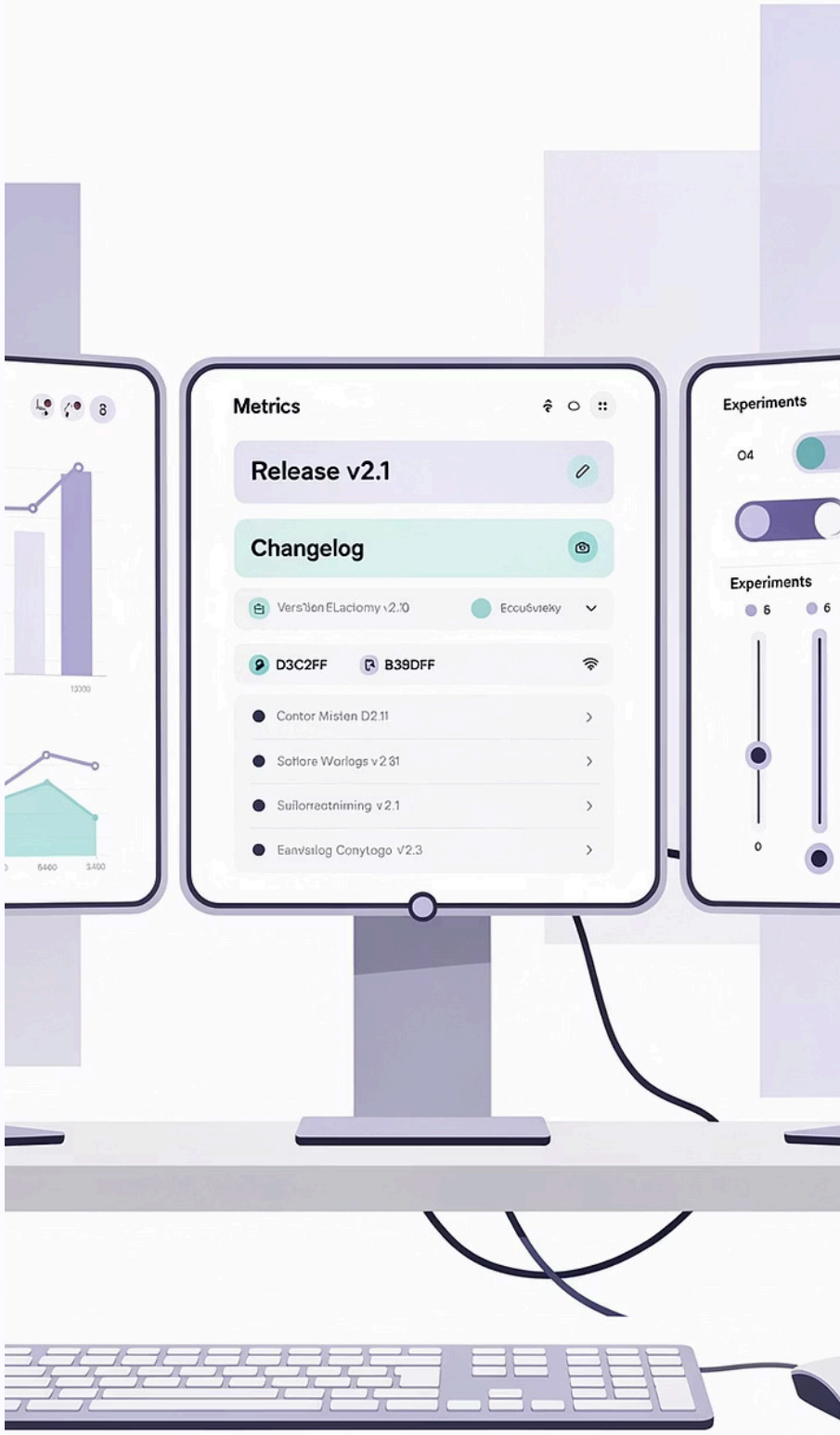
Register models with semantic versions and lifecycle stages — Staging, Production, Archived — enabling controlled promotion workflows.

Model Packaging

Package models with their dependencies using MLflow flavours (scikit-learn, PyTorch, XGBoost) ensuring reproducible deployments across environments.

Deployment

Serve models as REST endpoints or batch jobs. Integrates natively with Databricks, SageMaker, Azure ML, and local serving.



Feature Engineering at Scale

The secret to scalable feature engineering is treating features as first-class data assets — versioned, tested, and shareable across teams and models.

Delta Tables as Feature Stores

Delta Lake provides ACID transactions, time travel, and schema evolution — making it an ideal foundation for feature tables. Store pre-computed features at the grain models expect (per customer, per session, per device), and use Delta's `OPTIMIZE` and `ZORDER` to ensure fast point-in-time lookups at serving time.

> Always materialise features using a deterministic timestamp column to prevent training-serving skew — one of the most common sources of silent model degradation in production.

Reusable Feature Definitions

Define features once, consume many times. Reusable feature pipelines mean:

- New models onboard in hours, not weeks
- Consistent business logic across churn, propensity, and fraud models
- Centralised backfill and refresh schedules
- Lineage tracked from raw source to model prediction

Pipeline Integration: How It All Connects

A fully integrated stack chains Delta Live Tables, SQL analytics, and ML model training into a single declarative workflow — each layer building on the last.

DLT → Features

Delta Live Tables ingest and transform raw data into clean, validated feature tables. Declarative pipelines enforce data quality expectations automatically.

SQL → Analysis

Analysts and engineers query feature tables directly via SQL for exploratory analysis, cohort building, and labelling — feeding signal back into feature design.

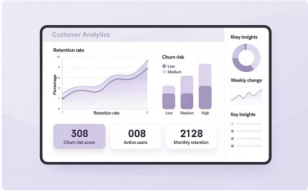
ML → Models

ML engineers read from feature tables, train models tracked in MLflow, and register them for batch or real-time scoring — closing the production loop.

This architecture eliminates bespoke glue code between each stage. Changes propagate through the pipeline automatically, and every artefact — from raw event to deployed model — is traceable.

Real-World Examples

These two use cases demonstrate the full integration pattern in practice — from raw event data through to a deployed, monitored model.



Customer Churn Model

Ingest CRM and product usage events via DLT → compute recency, frequency, and engagement features in Delta → train a gradient-boosted classifier tracked in MLflow → serve churn probability scores nightly to the CRM for proactive outreach. Feature tables are shared with the marketing personalisation model, eliminating duplicate engineering.

Fraud Detection

Stream transaction events into DLT with millisecond latency → engineer velocity features (transactions per hour, geo-distance from last purchase) in near real-time Delta tables → deploy a low-latency XGBoost model via MLflow REST endpoint → score transactions inline at checkout. Model drift is monitored against a rolling baseline and triggers automated retraining when thresholds are breached.

You Can Now Build Full Data + ML Systems

Across this session, you've moved from fragmented pipelines to a unified, production-ready architecture. Every piece connects deliberately.

01

Unified Platform

Data, ML, and governance on a single stack — Delta Lake, MLflow, and Unity Catalog working together

02

Reusable Features

Delta-backed feature tables shared across models, with lineage, versioning, and quality built in

03

Integrated Pipelines

DLT → SQL → ML as a single declarative workflow, from raw ingestion to deployed model endpoint

04

Production Patterns

Proven on churn prediction and fraud detection — now apply the same pattern to your next model

Your next step: Identify one existing model in your organisation. Map its data dependencies, extract the feature logic into a shared Delta table, and register it in MLflow. That single change delivers immediate, tangible value.